Weekly Student Progress Performa

Team Id	CS26084	**Week no / Date**	Week 7 31 August 2026
Project Title	SentinelCore: A Network Detection and Incident Response Platform		
Members Present	Om Rashiya (24cs084), Yash Patel (24cs082)		
Mentor	Akshita Kadam		

WEEK 7 • Event Ingestion and Search Experience

Section	Details to be filled by the team
Planned Tasks (from last week)	Create a resilient event-ingestion pipeline from Suricata EVE JSON to PostgreSQL.
Tasks Completed This Week	Built a dedicated worker, EVE tailer, event normalizer, batch writer, Redis-persisted read offsets, monthly PostgreSQL partitions, retention utilities, pipeline status endpoints, and event model validation. Kept ingestion outside the API process to prevent duplicate readers.
Commits Made	Total: 0 standalone commits this review period. Pipeline work was reviewed continuously and included in the consolidated M1–M5 milestone commit d79a37c.
Issues Created / Closed	Created: Worker restarts could reread events or lose file position; multiple API replicas could duplicate ingestion. Closed: Moved tailing to one worker and persisted offsets in Redis with idempotent database writes.
Milestone Status	~54% complete — on track. Detection events could flow reliably from sensor output into partitioned storage.
Blockers / Challenges Faced	Handling log rotation, partial JSON lines, malformed events, and database backpressure without halting the pipeline.
Plan for Next Week	Implement fast event search, facets, saved searches, and rule-driven correlation.

Research and Technical Study

Research objective. Create a resilient event-ingestion pipeline from Suricata EVE JSON to PostgreSQL.

Study completed. Compared file-tail reliability patterns, at-least-once delivery, checkpointing, batch insertion, and time-based partitioning. The worker acknowledges progress only after durable writes, and malformed lines are isolated rather than stopping ingestion.

Event-pipeline reliability decisions

Condition	Failure risk	Implemented handling
Worker restart	Replay or skipped data	Redis-persisted inode and offset
Log rotation	Silent event loss	Drain old handle, then reopen
Malformed input	Pipeline interruption	Isolate the line and continue
Repeated event	Duplicate record	Deterministic deduplication key
Growing volume	Large-table slowdown	Monthly partitions and retention

Decision carried into the project

Built a dedicated worker, EVE tailer, event normalizer, batch writer, Redis-persisted read offsets, monthly PostgreSQL partitions, retention utilities, pipeline status endpoints, and event model validation. Kept ingestion outside the API process to prevent duplicate readers.

Implementation Record and Website Evidence

Dedicated ingestion

Placed EVE file reading in one worker instead of the request-serving API. This avoids competing readers and keeps long-running ingestion separate from interactive website traffic.

Restart and rotation handling

Tracked the EVE file identity and read position, persisted checkpoints in Redis after durable batches, and handled rotation so the worker can finish the old file before reopening the new one.

Normalization and storage

Mapped alert, flow, DNS, HTTP, and TLS records into a consistent event schema, retained the original record, generated a deduplication key, and wrote to monthly PostgreSQL partitions.

Event explorer

Prepared the website view with time-range controls, severity/type/protocol filters, address and text search, saved searches, live tail, a results table, and facets.

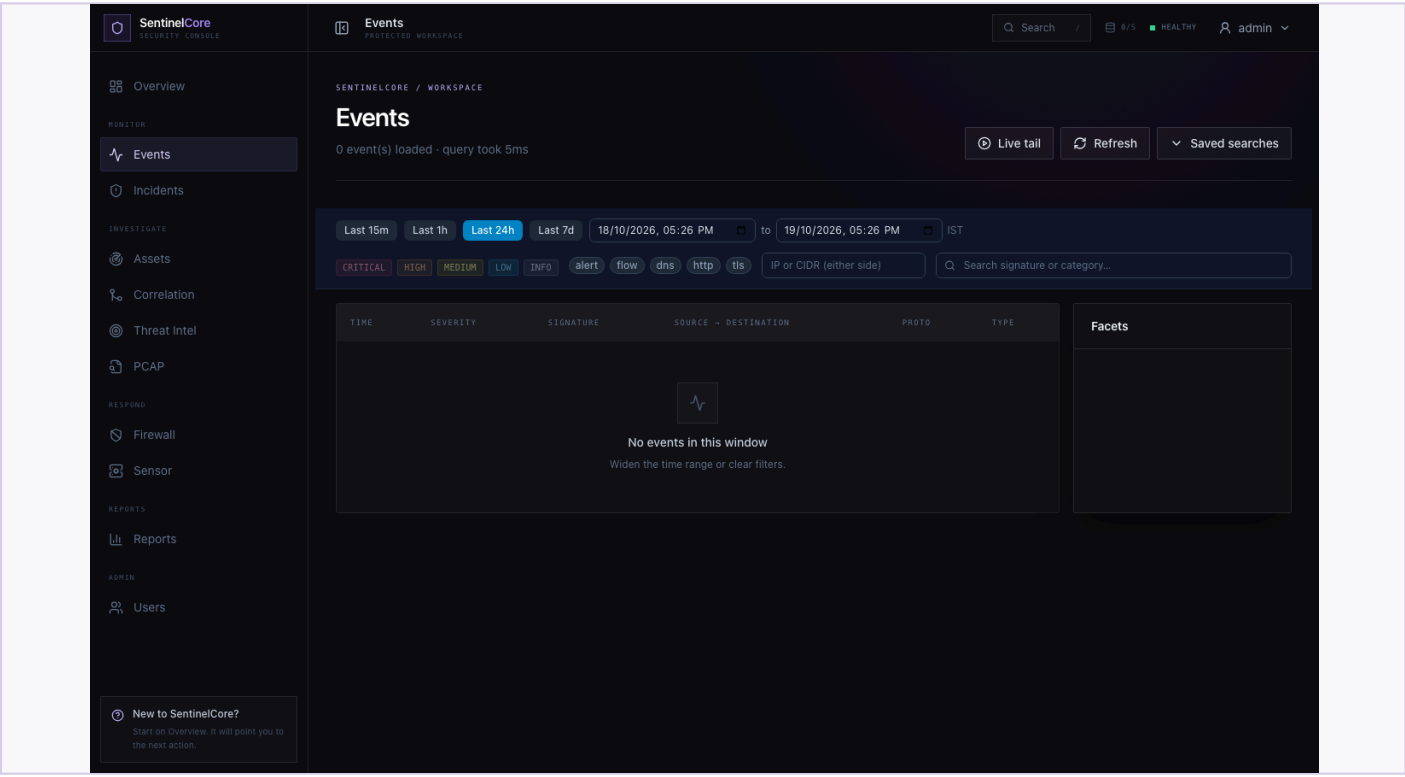


Figure W7.1 — Project-owned SentinelCore interface mapped specifically to Week 7.

Evidence interpretation. The Events workspace exposes time windows, severity chips, event-type and protocol controls, address/text search, live-tail mode, saved searches, results, and facets. Its visible controls, labels, status fields, and navigation provide direct evidence for the work described on this page.

Repository and Architecture Evidence

EVE tailer and checkpoint implementation

backend/app/pipeline/tailer.py

```
58          )
59
60
61  class EveTailer:
62      def __init__(self, path: str | Path, redis: aioredis.Redis) -> None:
63          self.path = Path(path)
64          self.redis = redis
65          self._handle = None
66          self._identity: FileIdentity | None = None
67          self._offset = 0
68          self._stopping = False
69
70          self.lines_read = 0
71          self.parse_errors = 0
72          self.rotations = 0
73
74      # ------------------------------------------------------------------
75      # Offset persistence
76      # ------------------------------------------------------------------
77
78      async def _load_offset(self) -> tuple[int, FileIdentity | None]:
79          """Resume where the last run stopped, rather than replaying or skipping."""
80          try:
81              raw = await self.redis.get(OFFSET_KEY)
```

Repository evidence captured directly from the SentinelCore workspace

Figure W7.2 — EVE tailer and checkpoint implementation. This is a separate internal artifact, not a copy of the website image.

Repository files used for traceability

- reports/modules/M05-event-pipeline_1.md
- backend/app/pipeline/tailer.py
- backend/app/pipeline/normalize.py
- frontend/src/pages/events/EventsPage.jsx

Verification

Pipeline wording follows the implemented tailer, normalizer, writer, partitions, and Events page; no claimed event source outside Suricata/PCAP is introduced.

Traceability conclusion. EVE tailer and checkpoint implementation directly supports the Event Ingestion and Search Experience milestone. Pipeline wording follows the implemented tailer, normalizer, writer, partitions, and Events page; no claimed event source outside Suricata/PCAP is introduced. The artifact, source paths, implementation record, and website evidence therefore describe the same weekly scope.

Weekly Closure and Faculty Assessment

Completed outcome

Built a dedicated worker, EVE tailer, event normalizer, batch writer, Redis-persisted read offsets, monthly PostgreSQL partitions, retention utilities, pipeline status endpoints, and event model validation. Kept ingestion outside the API process to prevent duplicate readers.

Commit record

Total: 0 standalone commits this review period. Pipeline work was reviewed continuously and included in the consolidated M1–M5 milestone commit d79a37c.

Issue and resolution

Created: Worker restarts could reread events or lose file position; multiple API replicas could duplicate ingestion. Closed: Moved tailing to one worker and persisted offsets in Redis with idempotent database writes.

Milestone position

~54% complete — on track. Detection events could flow reliably from sensor output into partitioned storage.

Challenge handled

Handling log rotation, partial JSON lines, malformed events, and database backpressure without halting the pipeline.

Next-week hand-off

Implement fast event search, facets, saved searches, and rule-driven correlation.

Repository reference

https://github.com/rashiyaom/SentinelCore

Assessed By (Faculty)	Marks Awarded (5 marks)	Faculty Remarks